

LOS Integration Guide

LoanInNeed (LIN) ↔ Loan Origination System (LOS) — Technical & Business Reference

Document Purpose: This document explains — in plain business and technical terms — how the LoanInNeed platform (LIN) integrates with the external Loan Origination System (LOS). It is intended for both business stakeholders and developers. It also clearly lists what information is still required from the LOS team before the integration can go fully live.

1. What Is This Integration?

When a customer submits a loan application on the **LoanInNeed website**, that application needs to be sent to the **LOS system** (the LOS team's platform) so they can review it, verify the customer, and process the loan.

Without this integration:

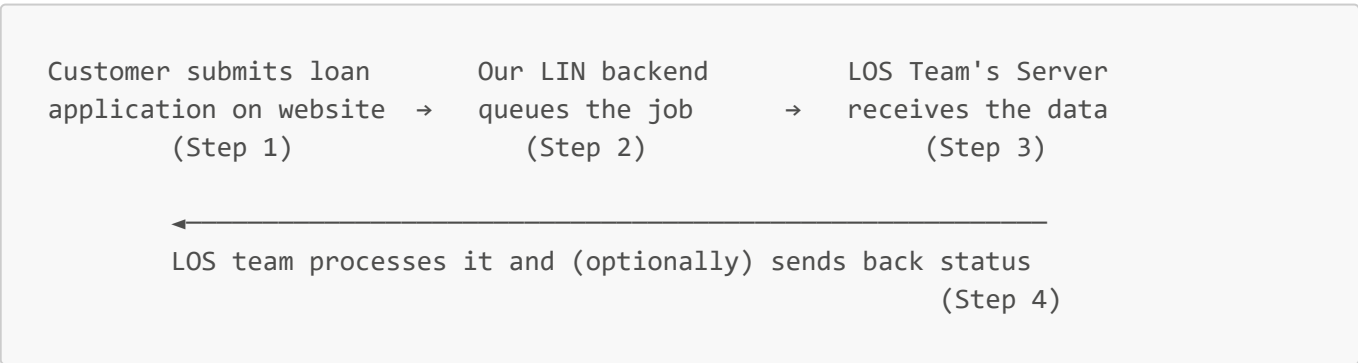
- LIN captures the customer's loan application
- The LOS team would never receive it automatically — they would have to manually pull data from us

With this integration:

- As soon as a customer applies, LIN automatically pushes the application to LOS in real time
- The LOS team receives the data instantly, without any manual effort
- LIN records whether the push succeeded or failed, and automatically retries on failure

This is known as a **push-based, real-time API integration**.

2. How the Data Flows (Architecture)



Step-by-step Flow

Step	Who Acts	What Happens
1	Customer	Fills and submits the loan application form
2	LIN Backend	Creates the application in the LIN database + adds a "Push to LOS" job to the queue

Step	Who Acts	What Happens
3	LIN Background Worker	Runs every minute, picks up queued jobs, and pushes them to LOS via HTTP POST
4	LOS System	Receives the data, returns a referenceId confirming receipt
5	LIN Backend	Saves the referenceId to track the application on the LOS side
6	LOS Team (optional)	Can call back to LIN's update endpoint to push status changes (APPROVED / REJECTED)

3. What Data Gets Sent to LOS?

Each application push includes the following information:

```
{
  "applicationId": "LIN/2026/00021",
  "customer": {
    "firstName": "Rahul",
    "lastName": "Sharma",
    "mobile": "9876543210",
    "email": "rahul@example.com"
  },
  "loanDetails": {
    "product": "Payday",
    "amount": 50000,
    "tenure": 30,
    "emiType": "Single EMI"
  },
  "source": "WebApp",
  "timestamp": "2026-04-06T10:30:00Z"
}
```

Field Reference

Field	Description	Source
applicationId	Unique LIN application number in LIN/YYYY/NNNNN format	Auto-generated by LIN
customer.firstName	Customer's first name	Extracted from profile on LIN
customer.lastName	Customer's last name	Extracted from profile on LIN
customer.mobile	Registered mobile number	LIN user record
customer.email	Registered email address	LIN user record
loanDetails.product	Loan product type (e.g. "Payday")	Derived from loan type on LIN

Field	Description	Source
<code>loanDetails.amount</code>	Requested loan amount (₹)	Entered by customer on LIN
<code>loanDetails.tenure</code>	Loan duration in days (default 30)	LIN default for payday loans
<code>loanDetails.emiType</code>	Repayment type (default "Single EMI")	LIN default for payday loans
<code>source</code>	Origin platform	Always "WebApp" for web submissions
<code>timestamp</code>	Exact time of push (UTC)	Server time

4. How Authentication Works

LIN sends every request to LOS with two security credentials in the HTTP headers:

Header	Value	Description
<code>Authorization</code>	<code>Bearer <token></code>	A secret bearer token provided by the LOS team
<code>x-api-key</code>	<code><api_key></code>	A static API key provided by the LOS team
<code>Content-Type</code>	<code>application/json</code>	Always JSON format

⚠ **These credentials must be provided by the LOS team.** See [Section 6: What We Need From the LOS Team](#).

5. What LOS Returns (Response Handling)

On Success

LOS responds with a confirmation that includes a `referenceId` — their internal tracking number for the application:

```
{
  "status":      "SUCCESS",
  "message":     "Application received successfully",
  "referenceId": "LOS123456",
  "timestamp":   "2026-04-06T10:30:02Z"
}
```

LIN saves this `referenceId` to our database so we can:

- Track which LOS record corresponds to which LIN application
- Use it for any future status update communication

On Failure

If LOS rejects the request:

```
{
  "status":    "FAILED",
  "errorCode": "VALIDATION_ERROR",
  "message":   "Missing required fields"
}
```

LIN will log the error and **automatically retry up to 3 times** before marking the job as permanently failed and alerting the team.

6. What We Still Need From the LOS Team

● **The integration code is fully written and ready. All the items below are blockers before going live. Please share the following with us:**

6.1 — LOS Server URL (Critical)

We need the **base URL of the LOS server** — this is where we will send the application data.

```
Expected endpoint:
POST https://<LOS_SERVER_URL>/api/v1/loan/applications
```

What to ask for: *"What is the base URL of your LOS server?"*

Example of what it should look like: <https://los.yourcompany.com> or <https://api.los.in>

6.2 — Bearer Token (Critical)

We need the **Bearer token** for the **Authorization** header.

What to ask for: *"What is the Bearer token we should use in the Authorization header when calling your API?"*

This is a secret string that should be treated like a password and shared securely (not over email).

6.3 — API Key (Critical)

We need the **API key** for the **x-api-key** header.

What to ask for: *"What is the x-api-key value we should include in our request headers?"*

6.4 — Status Update Endpoint (Important but Not Blocking)

Your document shows only the **application submission** endpoint (**POST** [/api/v1/loan/applications](#)). However, you mentioned:

"push the status of the Loan application based on the requirement"

For us to push status updates (e.g., when our admin marks an application as APPROVED or REJECTED), we need:

What to ask for:

"Do you have an endpoint for us to push loan status updates? If yes, please share:

- *The full endpoint URL*
- *The HTTP method (POST / PUT / PATCH)*
- *The expected request body format*
- *Any additional headers*

Example of what this might look like:

```
PUT https://<LOS_SERVER_URL>/api/v1/loan/applications/{referenceId}/status
Body: { "status": "APPROVED", "remarks": "All documents verified", "timestamp":
"..."}

```

6.5 — Environment (Staging vs. Production)

What to ask for: *"Do you have a staging/sandbox server we can test against before going to production?"*

This allows us to do a full end-to-end test without affecting real data.

6.6 — IP Whitelist (If Required)

Some LOS systems require the calling server's IP address to be whitelisted.

What to ask for: *"Do we need to whitelist our server's IP address to access your API? If yes, let us know and we will share our server IP."*

7. What LIN Has Already Built

Everything on our side is **fully implemented and ready**:

Component	Status	Description
Application submission queue	☒ Done	Every loan application automatically gets queued for LOS push
Background cron worker	☒ Done	Runs every minute, picks up queued jobs, sends to LOS
Retry logic	☒ Done	Automatically retries up to 3 times on failure
New payload builder (<i>buildNewLosPayload</i>)	☒ Done	Builds exactly the JSON format LOS expects

Component	Status	Description
Auth headers	Done	Sends Bearer token + x-api-key — just needs the actual values
Response handling	Done	Reads referenceId from LOS response and stores it
Status update placeholder	Done	Code is ready; awaiting LOS status endpoint URL
Pull endpoints for LOS	Done	LOS can also pull data from us at /api/los/applications
Callback endpoint for LOS	Done	LOS can call us at PUT /api/los/applications/:id/status to update status
.env configuration	Done	Placeholder variables set; just needs real values from LOS team

8. What the LOS Team Can Do on Their End (Endpoints We Expose)

In addition to pushing data to LOS, we also expose endpoints that LOS can call:

8.1 — Pull All Applications (GET)

LOS can fetch all loan applications from LIN at any time.

```
GET https://lionfish-app-mg3te.ondigitalocean.app/api/los/applications?
status=PENDING
Authorization: Key paromita$432
```

8.2 — Pull Detailed Export (GET)

Full details including KYC, documents, and employment info.

```
GET https://lionfish-app-mg3te.ondigitalocean.app/api/loans/export?from=2024-01-
01&to=2026-12-31
Authorization: Key paromita$432
```

8.3 — Push Status Update to LIN (PUT — Callback Webhook)

When LOS processes an application (approves, rejects, disburses), they can call this to update the status in our system.

```
PUT https://lionfish-app-
mg3te.ondigitalocean.app/api/los/applications/{applicationId}/status
```

```
Authorization: Key paromita$432
Content-Type: application/json

Body:
{
  "status": "COMPLETED",
  "losApplicationId": "LOS-912",
  "losCaseNumber": "CASE-123",
  "losKycId": "KYC-099",
  "lastError": ""
}
```

9. Summary Checklist

For Our Team (LIN)

- ☒ Payload format updated to match LOS contract
- ☒ Auth headers implemented (Bearer + x-api-key)
- ☒ Response parsing updated for **referenceId**
- ☒ Application ID formatted as **LIN/YYYY/NNNNN**
- ☒ Retry logic: up to 3 retries, then alerts team
- ☒ Status push code ready (waiting for LOS endpoint)
- ☐ **Fill in **.env** with real LOS credentials once received**
- ☐ **Test against LOS staging server**
- ☐ **Go live on production**

Still Needed From LOS Team

- ☐ LOS server base URL
- ☐ Bearer token
- ☐ API key (**x-api-key**)
- ☐ Status update endpoint (URL + method + payload format)
- ☐ Staging/sandbox URL (for testing)
- ☐ Confirm if IP whitelisting is needed

10. Contact & Escalation

Role	Responsibility
LIN Backend Team	Will fill .env once credentials are received
LOS Team	Must share credentials and status endpoint

Once all credentials are received, estimated time to go live: **Same day** (no code changes needed, just **.env** values).

Document prepared by: LIN Backend Team

Last updated: April 2026